

Nataraja — Architecture Study Guide

Purpose. Sit down, read this top-to-bottom, and you will understand the *complete* platform — what every piece is, why it exists, how it fits, and the trade-offs behind each choice. It's split into **16 study segments**. Each segment has the same shape so it's easy to revise:

- **What it is** — plain English.
- **Why it exists** — the problem it solves.
- **How it works** — the mechanism / the flow.
- **Key concepts** — the terms to master.
- **Decisions & trade-offs** — the "why this, not that" (this is what interviews probe).
- **Explain-it-simply** — one sentence a non-technical person understands.

Companion material: [notes.html](#) (concept lessons with diagrams), [architecture.html](#) (interactive diagram), [LEARNING-PLAN.md](#) (the build calendar). This guide is the *narrative* that ties them together.

Segment 0 — The big picture & the request lifecycle

What it is. Nataraja is an asynchronous **job-orchestration platform / LLM gateway**. Clients submit "jobs" (e.g., "run this agent task"); the platform accepts them instantly, runs them reliably on a worker fleet against LLM providers, and stores every result — multi-tenant, resilient, observable.

Why it exists. Calling an LLM directly from a web request is fragile: the call is slow, can fail, has rate limits, and costs money. A gateway/orchestrator absorbs all that complexity so clients get a fast, reliable, billed, observable service instead of a raw, flaky API call.

The request lifecycle (memorize these 12 steps — it's your whiteboard story):

1. Client hits the **edge** (Cloudflare CDN → WAF → DNS → TLS → ingress).
2. Request reaches the **FastAPI gateway**.
3. Gateway **authenticates** the caller (Keycloak/JWT) and checks **quota**.
4. Gateway writes a job row (**Postgres**), returns **202 + job id immediately** (never makes the user wait).
5. The job is published to the **event backbone** (Kafka) — the "accept" path ends here.
6. A **Go worker** picks the job off Kafka.
7. The worker runs the **agent loop** (plan → act → observe) against **Claude**, wrapped in the **resilience layer** (rate-limit, breaker, retry, fallback).
8. Each step is recorded as an **event** (event sourcing); the trajectory goes to **MongoDB**.
9. Output **streams back** to the browser live via **SSE**.
10. Final state + usage are written to **Postgres** (ledger) and analytics flow to the **warehouse** (via CDC).

11. Everything runs on **Kubernetes**, deployed by **GitOps** (Argo CD).

12. **Observability** (OTel/Prometheus/Loki/Grafana) watches the entire path against **SLOs**.

The "planes" mental model. Group the parts into planes: **Edge**, **Control** (routing/identity/quota), **Data plane** (the request path: gateway → Kafka → workers → providers), **Storage plane** (polyglot DBs), **Platform plane** (K8s/Terraform/Vault), **Observability plane**. Cross-cutting planes wrap everything.

Explain-it-simply. *It's a smart post office for AI tasks: you drop a task, get a ticket instantly, and reliable workers do it in the background while you watch progress live.*

Segment 1 — Edge & Entry

What it is. Everything between the user's browser and your cluster: **Cloudflare CDN**, **AWS WAF**, **Route 53 (DNS)**, **TLS** (ACM/cert-manager), and the **ingress controller**.

Why it exists. You need traffic to arrive **fast**, **encrypted**, **filtered of attacks**, and **routed to the right service** — before it ever touches your app.

How it works. DNS resolves your domain → traffic hits Cloudflare (caches static assets, blocks DDoS) → WAF filters malicious patterns → TLS is terminated (HTTPS) → ingress routes by host/path into the cluster.

Key concepts. CDN & edge caching; WAF rules (OWASP); DNS + health-based failover; TLS termination & auto-renewal (ACME); ingress controllers; the difference between L4 (TCP) and L7 (HTTP) load balancing.

Decisions & trade-offs. *CDN adds a vendor and cache-invalidation care, but buys global latency + DDoS protection cheaply. cert-manager automates certs (no expiry outages) but couples TLS to the cluster.*

Explain-it-simply. *The security gate and reception desk of the building — checks IDs, keeps the line moving, and points each visitor to the right room.*

Segment 2 — API Gateway & the "accept" path

What it is. The **FastAPI** service that is the platform's front door. Its job is to **accept work instantly**, not to do the work.

Why it exists. LLM jobs are slow (seconds to minutes). If the HTTP request waited for the job to finish, connections would pile up and the system would collapse. So we **separate accept from process**: accept in milliseconds, process in the background.

How it works. Request arrives → validate the body (**Pydantic**) → authenticate → check quota → write a job record → **return 202 Accepted + a job id** → publish the job to Kafka. The client later polls or streams for the result using the id.

Key concepts. **202 Accepted** pattern; **idempotency keys** (safe retries — the same request never creates two jobs); **optimistic concurrency control (OCC)** (two writers can't silently clobber each other); correlation IDs (trace one request across services); 12-factor config.

Decisions & trade-offs. *Async-accept adds complexity (clients must handle "not done yet") but is the only way to stay responsive under slow work. Idempotency costs a little storage but prevents double-charging on retries.*

Explain-it-simply. *The ticket counter: it takes your order and hands you a numbered ticket in seconds — it doesn't cook your food at the counter.*

Segment 3 — The async job core (queue, worker, state machine, event sourcing)

What it is. The heart of the background system: a **durable queue**, a **worker** that consumes it, an explicit **job state machine**, and an **append-only event log**.

Why it exists. Work must survive crashes, be retried safely, and have a **provable history** ("what happened to job X and when?").

How it works. The gateway enqueues a job. A worker pulls it and moves it through defined states (`pending` → `running` → `done/failed`). Every transition is written as an **event** (`JobCreated` , `JobStarted` , `JobFailed` ...). The current state is *derived by replaying the events* (**event sourcing**), so you never lose the story of what happened.

Key concepts. Durable queue (Redis Streams → later Kafka); **state machine** (only legal transitions allowed); **at-least-once delivery** + **idempotent consumers** (a message may arrive twice; processing it twice must be safe); **cancellation**; the **exactly-once "myth"** (true exactly-once delivery is impossible; you achieve *effectively-once* via idempotency).

Decisions & trade-offs. *Event sourcing gives a perfect audit trail and time-travel debugging, but you must rebuild state from events (more moving parts) — worth it for a system where "why did this job fail?" must always be answerable.*

Explain-it-simply. *A kitchen order ticket that records every step — received, cooking, plated — so you can always see exactly where any order is and what happened to it.*

Segment 4 — Polyglot persistence (the right database for each job)

What it is. Deliberately using **several databases**, each for what it's best at: **PostgreSQL** (jobs/tenants/quotas/ledger), **MongoDB** (agent trajectories/payloads), **Redis** (cache, hot state, counters), a **vector DB** (pgvector/Qdrant, for RAG), **object storage** (S3/MinIO, for big artifacts), and an **OLAP warehouse** (ClickHouse, for analytics).

Why it exists. No single database is good at everything. Forcing all data into one type means either rigid schemas where you need flexibility, or weak guarantees where you need ACID.

How it works. The relational **source of truth** (money, state) lives in Postgres with ACID guarantees. Variable-shaped, deeply-nested agent data lives in Mongo. Microsecond lookups and rate-limit counters live in Redis. "Find similar by meaning" lives in the vector DB. Large blobs live in object storage. Fast aggregations for dashboards live in ClickHouse.

Key concepts. ACID vs BASE; **embed vs reference** (Mongo modeling); indexing/partitioning/replication (Postgres); TTL indexes & change streams (Mongo); cache invalidation (Redis); embeddings & similarity search (vector); columnar storage (OLAP).

Decisions & trade-offs. *Polyglot = more operational surface, but each store does one thing excellently.* The interview line: *"I put money in Postgres for ACID, trajectories in Mongo for schema flexibility, and counters in Redis for speed — right tool per shape of data."*

Explain-it-simply. *You don't keep milk, tools, and photos in the same box — you use a fridge, a toolbox, and an album. Same idea for data.*

Segment 5 — The agent layer (the "intelligence")

What it is. The part that actually runs an **LLM agent**: a loop that lets Claude **plan** → **call a tool** → **observe the result** → **repeat** until the task is done — grounded by **RAG** and bounded by a **token budget**.

Why it exists. This is the product. A single LLM call can answer a question; an **agent** can *do a task* (search, compute, fetch, decide) by using tools in a loop.

How it works. The worker sends the goal to Claude. Claude replies either with an answer or a **tool call**. The worker runs the tool, feeds the result back, and loops. **RAG** injects relevant company data (found by vector search) so answers are grounded. A **token budget** caps cost/length so an agent can't loop forever. Output **streams** to the user via SSE. The full **trajectory** is stored.

Key concepts. Agent loop (plan-act-observe); **tool / function calling**; **context-window management** & token budgeting; **RAG** (embeddings → retrieve → augment prompt); multi-agent patterns (orchestrator-worker, evaluator-optimizer); safe loop termination; prompt-injection defense; eval harness.

Decisions & trade-offs. *Agents are powerful but can loop, cost, and go off-track — so you bound them (token budget), ground them (RAG), and record them (trajectories) for debugging and evals.*

Explain-it-simply. *A junior assistant who can use tools: you give a goal, they look things up, take steps, and report back — but with a spending limit and a written log of everything they did.*

Segment 6 — Resilience (keeping it alive when providers misbehave)

What it is. A protective wrapper around every provider call: **rate limiter**, **circuit breaker**, **retry with backoff + jitter**, **fallback chain**, and **load shedding**.

Why it exists. External LLM providers are slow, rate-limited, and sometimes down. Without protection, one struggling provider drags your whole platform down with it.

How it works. Before calling a provider: the **rate limiter** (token bucket in Redis) ensures you stay within limits. The **circuit breaker** stops calling a provider that's already failing (and probes periodically to see if it recovered). **Retries** re-attempt transient failures, waiting longer each time with **jitter** (randomness) so all workers don't retry in sync. If it still fails, a **fallback** tries a backup provider. Under extreme overload, **load shedding** drops excess requests to protect the rest.

Key concepts. Token bucket vs sliding window; breaker states (closed/open/half-open); exponential backoff + **jitter** (prevents the "thundering herd"); fallback as an anti-pattern (it can hide root causes); **load shedding vs backpressure**; distributed rate limiting.

Decisions & trade-offs. *Retries improve success but can amplify load if naive — jitter fixes that. Fallback keeps you serving but can mask real problems — use it deliberately. Your interview edge: "I've operated these in Envoy; here's how I built them in code."*

Explain-it-simply. *Circuit breakers and backup generators for your app — when one supplier fails, you don't blow a fuse; you slow down, retry smartly, or switch suppliers.*

Segment 7 — Event-driven backbone (Kafka)

What it is. **Kafka** (Redpanda locally) as the central, durable **event log** that decouples the fast gateway from the slower workers — plus the patterns around it (schema registry, outbox, DLQ, CQRS, Saga, CDC).

Why it exists. A durable log lets producers and consumers work at different speeds, lets you **replay** history, and lets many independent services react to the same events without tight coupling.

How it works. Producers append events to **topics** split into **partitions** (for parallelism). **Consumer groups** share the work; each partition is read in order. Failed messages go to a **dead-letter queue**. The **outbox pattern** guarantees a DB write and its event publish happen atomically. **CQRS** keeps separate write- and read-optimized models. **Saga** coordinates multi-step jobs with compensation on failure. **CDC (Debezium)** streams DB changes into the warehouse.

Key concepts. Topics/partitions/offsets; consumer groups; ordering guarantees; the **exactly-once myth** (achieve effectively-once via idempotency + offsets); schema evolution; outbox; DLQ; CQRS; Saga (orchestration vs choreography); change data capture.

Decisions & trade-offs. *Kafka is operationally heavy but gives durability, replay, and decoupling no simple queue can. CQRS adds a read model to maintain but makes reads fast and writes clean.*

Explain-it-simply. *A shared, permanent conveyor belt: anyone can put items on it or read from it at their own pace, and you can always rewind to see what came before.*

Segment 8 — Multi-tenancy & security

What it is. Serving many customers ("tenants") from one platform safely: **tenants**, **API keys**, **quotas**, a **usage ledger**, real **identity** (Keycloak / OAuth2 / OIDC / JWT), secrets via **Vault**, and RBAC.

Why it exists. Multiple customers share the infrastructure, so you must **isolate** them (one tenant can't see or starve another), **authenticate** callers, and **bill** usage.

How it works. Every request carries a token; **Keycloak** issues and validates it (who you are + what you can do). A tenant id scopes all data and enforces **quotas** (fair sharing). A **usage ledger** records consumption for billing. **Vault** issues short-lived DB credentials and certificates instead of long-lived passwords in config. **Prompt-injection defenses** protect the agent layer from malicious input.

Key concepts. OAuth2 / OIDC / JWT flows; tenant isolation & fair scheduling (shuffle-sharding); dynamic secrets & PKI; RBAC; encryption in transit/at rest; OWASP API Security; prompt injection.

Decisions & trade-offs. *Self-hosted Keycloak avoids per-user SaaS fees but is more to operate. Dynamic secrets (Vault) are safer than static passwords but add a moving part.*

Explain-it-simply. *An apartment building: everyone has their own key and their own locked unit, a meter tracks each unit's usage for the bill, and the master keys are kept in a guarded safe.*

Segment 9 — Internal communication & Go

What it is. How internal services talk to each other efficiently: **gRPC + Protobuf**, a **worker rewritten in Go** (for cheap concurrency), and a **service mesh (Istio)** for security and traffic control.

Why it exists. REST/JSON is great at the edge but slow and loose for high-volume internal calls. And a worker fleet that runs thousands of concurrent agents benefits from Go's lightweight concurrency.

How it works. Services define strict contracts in **Protobuf** and call each other over **gRPC** (fast binary protocol, supports streaming). **Go** workers use **goroutines** (cheap threads) and **channels** to run many jobs concurrently with tiny memory. **Istio** adds mutual-TLS encryption and traffic-shaping (canary, retries, timeouts) *between* services without changing app code.

Key concepts. gRPC vs REST; Protobuf contracts; Go concurrency (goroutines, channels, `context`); service mesh; **mTLS**; sidecar proxy pattern.

Decisions & trade-offs. *gRPC is faster and typed but not human-readable (harder to debug by eye). Go for the worker = great concurrency, but a second language to maintain. A mesh gives free mTLS + traffic control but adds latency and complexity.*

Explain-it-simply. *Inside the company, staff use a fast internal phone system with a strict format, instead of writing polite public letters to each other.*

Segment 10 — Kubernetes & the custom Operator

What it is. The container platform that runs everything (**Kubernetes**, EKS/GKE), plus a **custom Operator** + **CRD** you write to automate platform tasks, and **KEDA** for event-driven autoscaling.

Why it exists. You need self-healing, scaling, and consistent deployment across many machines — and you want to automate your *own* operational knowledge (like provisioning a tenant) as native K8s objects.

How it works. K8s schedules and heals your containers. RBAC/NetworkPolicies/PodSecurity lock them down. A **custom Operator** watches a **CRD** (e.g., a `Tenant` or `PreviewEnv` resource) and runs a **reconcile loop** — continuously making reality match the declared desired state. **KEDA** scales workers based on **Kafka lag** (how many jobs are waiting), not just CPU. **Kyverno/OPA** enforce policy-as-code.

Key concepts. Pods/Deployments/StatefulSets; RBAC/NetPol/PodSecurity; **CRD + controller + reconcile loop**; the "desired state vs actual state" model; KEDA (scale on queue depth, scale-to-zero); admission control / policy-as-code; Helm & Kustomize.

Decisions & trade-offs. *Writing an Operator is real effort, but it turns manual ops into automated, self-healing behavior — and "I wrote a Kubernetes operator in Go" is a top-tier differentiator.*

Explain-it-simply. *A robot building manager: you write down "I want 3 tidy rooms," and it constantly checks and fixes things until reality matches — forever, without you.*

Segment 11 — Frontend & the experimentation framework

What it is. A **Next.js/React/TypeScript** dashboard (submit jobs, watch live trajectories, view health/usage) and an **A/B experimentation framework** (feature flags, traffic splitting, statistics).

Why it exists. Humans need a window into the system, and the business needs to change agent configs/models **safely and measurably**, not by guesswork.

How it works. The dashboard talks to the gateway and shows the **live trajectory** via SSE. The experimentation framework puts new agent configs behind **feature flags**, routes a **percentage of traffic** to them, measures outcomes, checks **statistical significance**, and **auto-rolls-back** if a guardrail metric drops.

Key concepts. React/Next/TypeScript basics; SSE/streaming UI; feature flags; traffic splitting; statistical significance & guardrails; auto-rollback.

Decisions & trade-offs. *An experimentation framework is extra machinery, but it turns risky "ship and pray" changes into data-driven, reversible decisions.*

Explain-it-simply. *A cockpit dashboard plus a way to test a new recipe on 5% of customers first — and instantly revert if they don't like it.*

Segment 12 — Per-tenant preview environments

What it is. On-demand, **isolated, disposable copies** of the stack — one per tenant/branch — created automatically and torn down when finished.

Why it exists. Teams and customers need a safe, real environment to try changes without touching production or each other.

How it works. Your Stage-10 **Operator** provisions an isolated stack (a **vcluster** or a dedicated namespace), wires up a **domain + TLS** (external-dns + cert-manager), and registers it via **Argo CD ApplicationSets** (GitOps). When done, it's torn down automatically.

Key concepts. Environment isolation (vcluster/namespace-per-tenant); external-dns + cert-manager; GitOps ApplicationSets; environment lifecycle & teardown; ephemeral infrastructure.

Decisions & trade-offs. *Preview envs cost resources while alive, so lifecycle/teardown must be automatic — but they massively speed up safe iteration.*

Explain-it-simply. *A pop-up showroom: spin up a full mini-version of the shop for one customer, then pack it away when they're done.*

Segment 13 — Cloud, networking & multi-region

What it is. The real cloud foundation, defined as code: **Terraform** provisioning AWS (VPC, EKS with IRSA, RDS, ElastiCache, MSK, S3, ALB, Route 53, CloudFront, ACM, WAF, KMS), evolving from single-region to **multi-region**, with **FinOps** cost control.

Why it exists. Production needs a secure network, managed data services, and (eventually) survival of a whole-region outage — all reproducible, reviewable, and cost-aware.

How it works. **Terraform** describes every resource as code (reviewable, repeatable). A **VPC** with public/private subnets isolates the network. **IRSA** gives pods least-privilege AWS access without static keys. Managed services (RDS, ElastiCache, MSK) reduce ops load. **Multi-region** starts **active-passive** (one region serves, another stands by with replicated data) and matures to **active-active** (both serve). **FinOps** (Infracost, budgets, tagging) keeps spend visible.

Key concepts. VPC/subnets/NAT; **SG vs NACL**; **IRSA** & IAM least-privilege; managed vs self-hosted; active-passive vs active-active; RPO/RTO (data-loss/recovery targets); FinOps.

Decisions & trade-offs. *Managed services cost more but save ops time. Active-active is the gold standard for availability but doubles cost and adds data-consistency complexity — justify it with real requirements.*

Explain-it-simply. *Building the actual property from a blueprint you can rebuild anytime — with a second identical site in another city in case the first floods.*

Segment 14 — Observability & SRE

What it is. Knowing whether the system is **healthy and fast**: **OpenTelemetry** (traces), **Prometheus** (metrics), **Loki** (logs), **Grafana** (dashboards), **Pyroscope** (profiling), plus **SLOs**, **chaos engineering**, and **supply-chain security**.

Why it exists. You can't operate what you can't see. And you need to define "good enough" reliability so alerts fire on **real user impact**, not noise.

How it works. **OTel** tags each request so you can follow it across every service (a **trace**). **Prometheus** collects metrics and fires alerts. **Loki** centralizes logs; **Grafana** visualizes it all. You define **SLOs** (e.g., 99.9% of requests succeed) with **error budgets**, and alert on **burn-rate** (spending the budget too fast).

Chaos Mesh injects failures on purpose to prove resilience. **SBOM/cosign/SLSA** prove your builds weren't tampered with.

Key concepts. The three pillars (traces/metrics/logs); instrumenting *your own code*; **SLO / SLI / error budget**; **burn-rate alerting**; chaos engineering; supply-chain security; continuous profiling.

Decisions & trade-offs. *Instrumentation is upfront effort but is the difference between "the site is slow, no idea why" and "trace shows the DB call at step 4 is the bottleneck."* SLO-based alerting cuts noise dramatically vs alert-on-everything.

Explain-it-simply. *The hospital monitors on the patient: heart rate, temperature, and a full chart — plus a rule that only pages the doctor when something actually matters.*

Segment 15 — Cross-cutting themes & interview prep

The recurring principles (say these and you sound senior):

- **Separate accept from process** — respond fast, work in the background.
- **Idempotency everywhere** — because everything retries.
- **The right tool per job** — polyglot persistence, gRPC internally / REST at the edge.
- **Design for failure** — breakers, retries, fallbacks, SLOs, chaos.
- **Everything as code & as data** — Terraform, GitOps, event sourcing, policy-as-code.
- **Observe what you build** — traces/metrics/logs/SLOs, not guesswork.

The "why" catalog (the questions interviewers ask):

- Why async accept + queue? → responsiveness under slow work.
- Why event sourcing? → provable history + rebuildable state.
- Why Kafka over a simple queue? → durability, replay, decoupling, fan-out.
- Why polyglot storage? → each store excels at one shape of data.
- Why jitter on retries? → prevents synchronized retry stampedes.
- Why an Operator? → automate ops as self-healing native objects.
- Why SLOs? → alert on user impact, protect against alert fatigue.

Capstone artifacts to produce (your portfolio proof): 30+ ADRs, a Well-Architected review, a cost model, C4 diagrams, DR runbooks, a failure playbook, and a scale-at-100x analysis.

Explain-it-simply. *Good architecture is mostly: be fast to say yes, assume things will fail, use the right tool for each job, and always be able to see and explain what's happening.*

How to study this (given limited time)

1. **First pass (1 sitting):** read Segment 0 + the "Explain-it-simply" line of every segment. Now you can describe the whole system.
1. **Second pass:** read each segment's *What / Why / How*. Now you understand it.
2. **Third pass:** read *Decisions & trade-offs* + Segment 15. Now you can *defend* it in an interview.
3. Pair each segment with its diagram in [architecture.html](#) and its concept lesson in [notes.html](#).

Reminder of the deal: this guide builds your **understanding**. The **code you write yourself**, stage by stage — that's what makes it defensible and yours. I coach and review; you build.